Chapter 8

Monitoring and Managing Linux Processes

- Listing Processes
  The **top** command in Linux is a powerful utility used to display a **dynamic, real-time** view of the **running system, including active processes, CPU usage, memory utilization, load averages,** and more.

```
[user@host ~]$ top
PID USER  PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
  1 root  20   0  244344  13684   9024 S   0.0   0.7   0:02.46 systemd
  2 root  20   0       0      0      0 S   0.0   0.0   0:00.00 kthreadd
...output omitted...
```

ps Stands for "**process status**" and provides a snapshot of active processes at the moment the command is run

```
[user@host ~]$ ps aux
USER       PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
...output omitted...
root         2  0.0  0.0      0     0 ?        S    11:57   0:00 [kthreadd]
student   3448  0.0  0.2 266904  3836 pts/0    R+   18:07   0:00 ps aux
...output omitted...
```

```
[user@host ~]$ ps aux
USER       PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root         1  0.1  0.1  51648  7504 ?        Ss   17:45   0:03 /usr/lib/systemd/
syst
root         2  0.0  0.0      0     0 ?        S    17:45   0:00 [kthreadd]
root         3  0.0  0.0      0     0 ?        S    17:45   0:00 [ksoftirqd/0]
root         5  0.0  0.0      0     0 ?        S<   17:45   0:00 [kworker/0:0H]
root         7  0.0  0.0      0     0 ?        S    17:45   0:00 [migration/0]
...output omitted...
[user@host ~]$ ps lax
F   UID   PID  PPID PRI  NI    VSZ   RSS WCHAN  STAT TTY      TIME COMMAND
4     0     1     0  20   0  51648  7504 ep_pol Ss   ?       0:03 /usr/lib/
systemd/
1     0     2     0  20   0      0     0 kthrea S    ?       0:00 [kthreadd]
1     0     3     2  20   0      0     0 smpboo S    ?       0:00 [ksoftirqd/0]
1     0     5     2   0 -20      0     0 worker S<   ?       0:00 [kworker/0:0H]
1     0     7     2 -100  -      0     0 smpboo S    ?       0:00 [migration/0]
...output omitted...
[user@host ~]$ ps -ef
UID        PID  PPID  C STIME TTY          TIME CMD
root         1     0  0 17:45 ?        00:00:03 /usr/lib/systemd/systemd --
switched-ro
root         2     0  0 17:45 ?        00:00:00 [kthreadd]
root         3     2  0 17:45 ?        00:00:00 [ksoftirqd/0]
root         5     2  0 17:45 ?        00:00:00 [kworker/0:0H]
root         7     2  0 17:45 ?        00:00:00 [migration/0]
...output omitted...
```

Name	Flag	Kernel-defined state name and description
Running	**R**	TASK_RUNNING: The process is either executing on a CPU or waiting to run. Process can be executing user routines or kernel routines (system calls), or be queued and ready when in the *Running* (or *Runnable*) state.
Sleeping	**S**	TASK_INTERRUPTIBLE: The process is waiting for some condition: a hardware request, system resource access, or signal. When an event or signal satisfies the condition, the process returns to *Running*.
	D	TASK_UNINTERRUPTIBLE: This process is also *Sleeping*, but unlike **S** state, does not respond to signals. Used only when process interruption may cause an unpredictable device state.
	K	TASK_KILLABLE: Identical to the uninterruptible **D** state, but modified to allow a waiting task to respond to the signal that it should be killed (exit completely). Utilities frequently display *Killable* processes as **D** state.
	I	TASK_REPORT_IDLE: A subset of state **D**. The kernel does not count these processes when calculating load average. Used for kernel threads. Flags TASK_UNINTERRUPTABLE and TASK_NOLOAD are set. Similar to TASK_KILLABLE, also a subset of state **D**. It accepts fatal signals.
Stopped	**T**	TASK_STOPPED: The process has been *Stopped* (suspended), usually by being signaled by a user or another process. The process can be continued (resumed) by another signal to return to *Running*.
	T	TASK_TRACED: A process that is being debugged is also temporarily *Stopped* and shares the same **T** state flag.
Zombie	**Z**	EXIT_ZOMBIE: A child process signals its parent as it exits. All resources except for the process identity (PID) are released.
	X	EXIT_DEAD: When the parent cleans up (*reaps*) the remaining child process structure, the process is now released completely. This state will never be observed in process-listing utilities.

Controlling Jobs

Job control is a feature of the shell which allows a single shell instance to run and manage multiple commands.

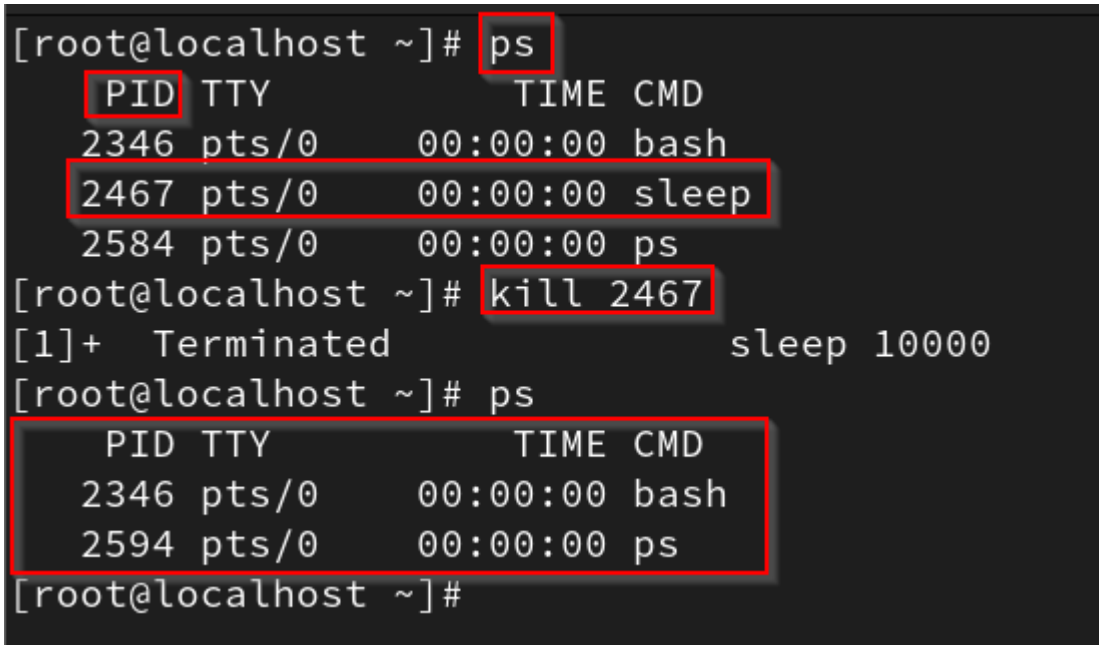
Running Jobs in the Background

```
[user@host ~]$ sleep 10000 &
[1] 5947
[user@host ~]$
```

You can **display the list of jobs** that Bash is tracking for a particular session with the **jobs** command.

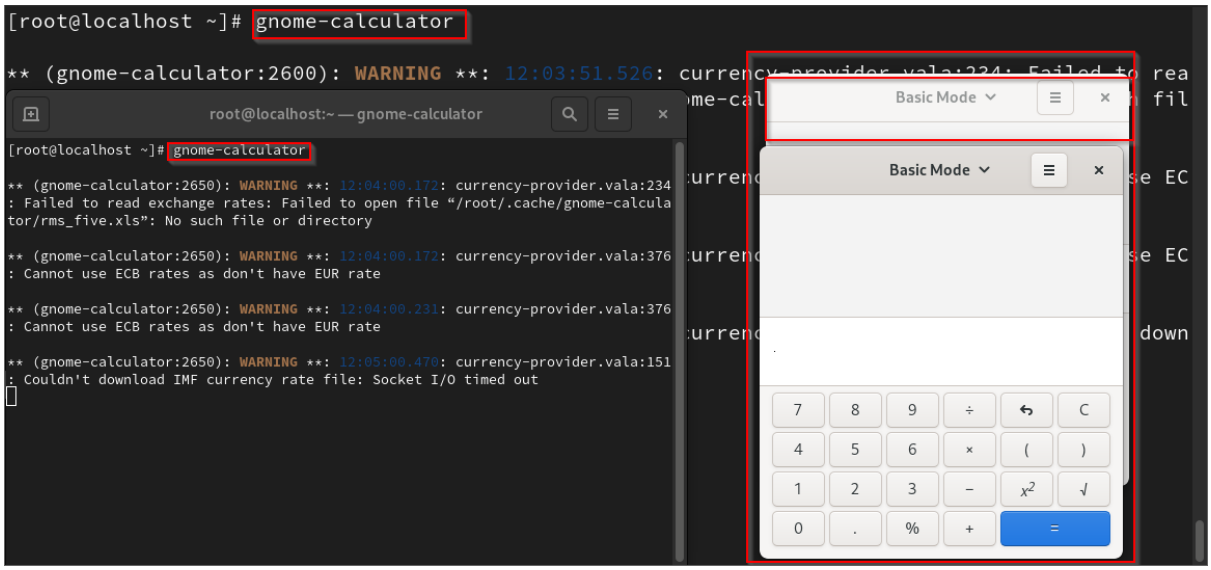
```
[user@host ~]$ jobs
[1]+  Running                 sleep 10000 &
[user@host ~]$
```

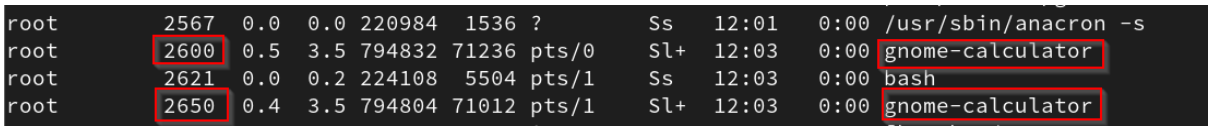
Killing Processes



To Create Multiple process

gnome-calculator

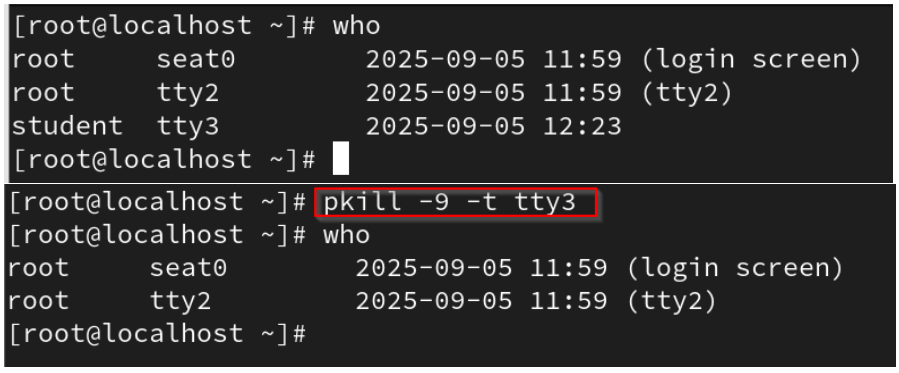




To kill all Same Processes **#killall "Common-process name"**

To kill All Process that is running by a user

To Kill all the terminal



Monitoring Process Activity

The **uptime** command is one way to display the current load average. It prints **the current time,how long the machine has been up**, **how many user sessions are running, and the current load average**

```
[root@localhost ~]# uptime
 12:28:36 up 29 min,  2 users,  load average: 0.00, 0.04, 0.07
[root@localhost ~]#
```

The **lscpu** command can help you **determine how many CPUs** a system has.

```
[root@localhost ~]# lscpu
Architecture:             x86_64
  CPU op-mode(s):         32-bit, 64-bit
  Address sizes:          45 bits physical, 48 bits virtual
  Byte Order:             Little Endian
CPU(s):                   1
  On-line CPU(s) list:    0
Vendor ID:                GenuineIntel
  BIOS Vendor ID:         GenuineIntel
  Model name:             Intel(R) Core(TM) i5-8500T CPU @ 2.10GHz
    BIOS Model name:      Intel(R) Core(TM) i5-8500T CPU @ 2.10GHz
    CPU family:           6
```